

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Bagas Aryo Dananjoyo - 5024241031

2026

1 Langkah-Langkah Percobaan

Bagian ini menjelaskan secara rinci tahapan atau prosedur yang dilakukan selama praktikum. Langkah-langkah ditulis secara urut dan sistematis, mulai dari persiapan alat hingga pelaksanaan percobaan. Penulisan harus jelas agar dapat dipahami oleh orang lain yang membaca laporan ini.

Pada praktikum Modul 1, dilakukan lima percobaan yang mencakup pengenalan board STM32 Blackpill dan ESP32, konfigurasi pull-up/pull-down resistor, debouncing, serta pembacaan ADC. Sebelum memulai, dilakukan persiapan lingkungan pengembangan yaitu instalasi Visual Studio Code beserta ekstensi PlatformIO, serta pembuatan satu project PlatformIO yang digunakan untuk seluruh percobaan pada modul ini. Setiap percobaan cukup mengganti isi platformio.ini dan file sumber (`src/main.cpp` atau `src/main.c`) sesuai kebutuhan.

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Tujuan: Memahami arsitektur board STM32 Blackpill dan melakukan flashing program Blink menggunakan ST-Link dengan framework Arduino.
Langkah Kerja:

1. Memastikan project Modul 1 sudah dibuat pada PlatformIO dan mengisi `platformio.ini` dengan konfigurasi berikut:

Listing 1: Konfigurasi platformio.ini untuk STM32 Blackpill

```
[env:blackpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

2. Menghubungkan ST-Link ke Blackpill menggunakan kabel jumper female-female dengan konfigurasi pin:

   - SWDIO $\leftrightarrow$ SWDIO (jalur data/debug)
   - SWCLK $\leftrightarrow$ SWCLK (jalur clock/debug)
   - GND $\leftrightarrow$ GND (referensi ground)
   - 3.3V $\leftrightarrow$ 3V3 (tegangan target board)

   Kemudian mencolokkan ST-Link ke komputer lewat USB.

3. Menulis kode program Blink pada file `src/main.cpp`:

Listing 2: Kode Program Blink STM32 Arduino

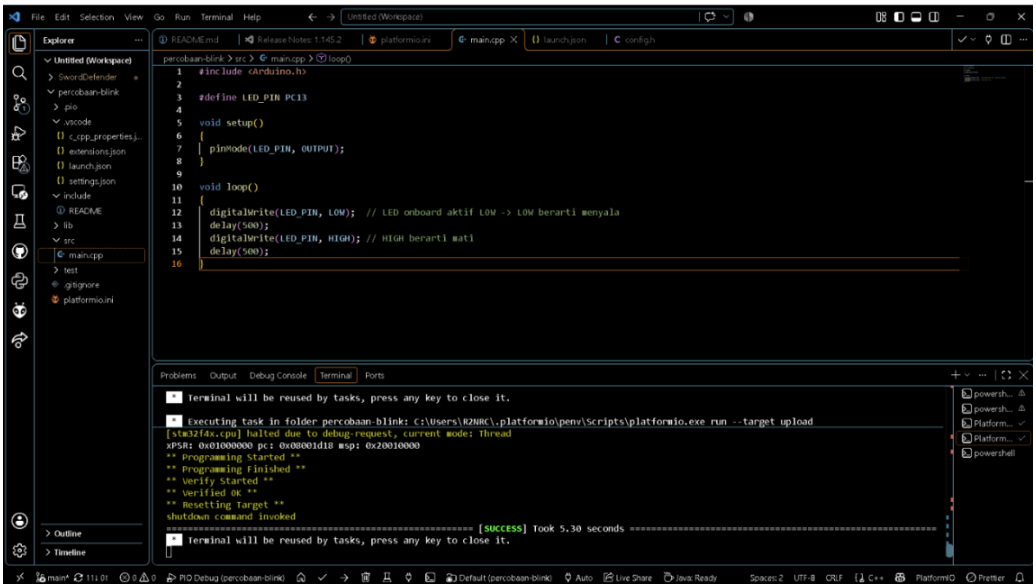
```
#include <Arduino.h>

#define LED_PIN PC13

void setup() {
    pinMode(LED_PIN, OUTPUT);
}
```

```
8
9  void loop() {
10     digitalWrite(LED_PIN, LOW); // LED onboard aktif LOW, LOW berarti menyala
11     delay(500);
12     digitalWrite(LED_PIN, HIGH); // HIGH berarti mati
13     delay(500);
14 }
```

4. Melakukan Build dan Upload melalui PlatformIO.
5. Mengamati LED onboard pada pin PC13: nyala selama 500ms, mati selama 500ms, berulang terus-menerus.



Gambar 1: Hasil Percobaan 1 - Program Blink STM32 Blackpill berhasil di-upload melalui ST-Link dan berjalan dengan status [SUCCESS]

1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

Tujuan: Memahami arsitektur ESP32 dan struktur program berbasis ESP-IDF, serta berhasil menjalankan program Blink menggunakan framework tersebut.

Langkah Kerja:

1. Mengganti isi `platformio.ini` dengan konfigurasi ESP32 dan framework ESP-IDF:

Listing 3: Konfigurasi platformio.ini untuk ESP32 ESP-IDF

```
1 [env:esp32dev]
2 platform = espressif32
3 board = esp32dev
4 framework = espidf
```

   Menunggu proses refresh/re-index konfigurasi PlatformIO hingga selesai sebelum melanjutkan.

2. Mencolokkan ESP32 DevKit ke komputer lewat USB dan memastikan port terdeteksi.
3. Menghapus file `src/main.cpp` dan membuat file baru `src/main.c`, kemudian menulis kode program Blink ESP-IDF:

Listing 4: Kode Program Blink ESP32 ESP-IDF

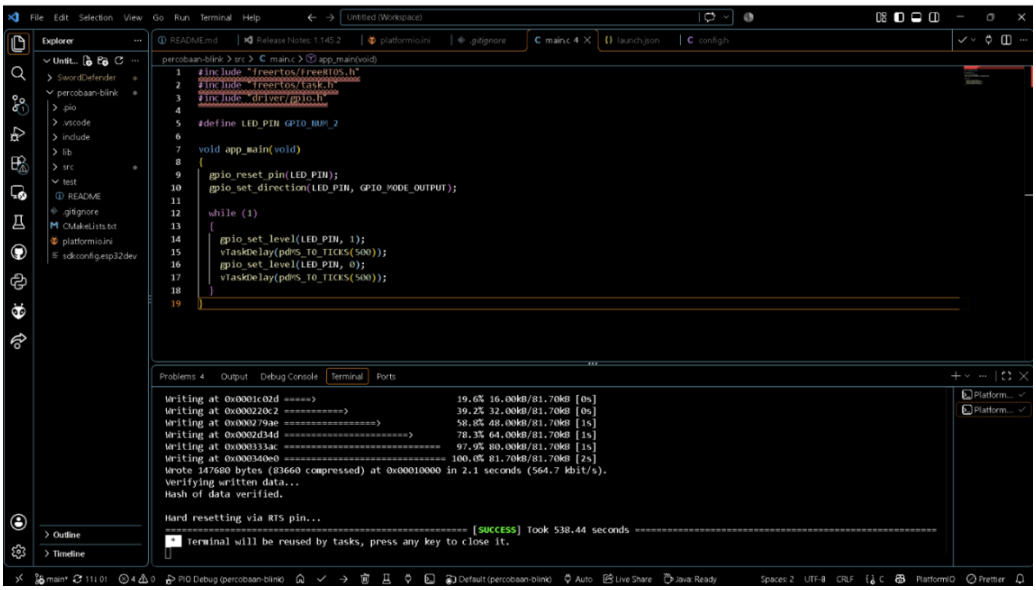
```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define LED_PIN GPIO_NUM_2

void app_main(void) {
    gpio_reset_pin(LED_PIN);
    gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);
    while (1) {
        gpio_set_level(LED_PIN, 1);
        vTaskDelay(pdMS_TO_TICKS(500));
        gpio_set_level(LED_PIN, 0);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

4. Melakukan Build dan Upload.

5. Mengamati LED onboard ESP32 (GPIO 2) berkedip dengan interval 500ms.



Gambar 2: Hasil Percobaan 2 - Program Blink ESP32 dengan framework ESP-IDF berhasil di-upload dan berjalan dengan status [SUCCESS]

1.3 Percobaan 3: Pull-up dan Pull-down - Eksternal vs Internal (ESP32 + Framework Arduino)

Tujuan: Memahami, mengimplementasikan, dan membandingkan pembacaan tombol menggunakan resistor pull-up/pull-down yang dipasang secara eksternal maupun diaktifkan secara internal pada ESP32 dengan framework Arduino.

Langkah Kerja:

1. Mengganti isi `platformio.ini` ke konfigurasi Arduino:

Listing 5: Konfigurasi platformio.ini untuk ESP32 Arduino

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
```

Menghapus `src/main.c` dan membuat kembali file `src/main.cpp`.

2. **Bagian A - Pull-up/Pull-down Eksternal:** Memasang dua pushbutton di GPIO 32 (pull-up eksternal dengan resistor $10\text{k}\Omega$ ke 3.3V) dan GPIO 33 (pull-down eksternal dengan resistor $10\text{k}\Omega$ ke GND).

3. Menulis Kode Program A (Eksternal) pada `src/main.cpp`:

Listing 6: Kode Program A - Pull-up & Pull-down Eksternal

```
#include <Arduino.h>

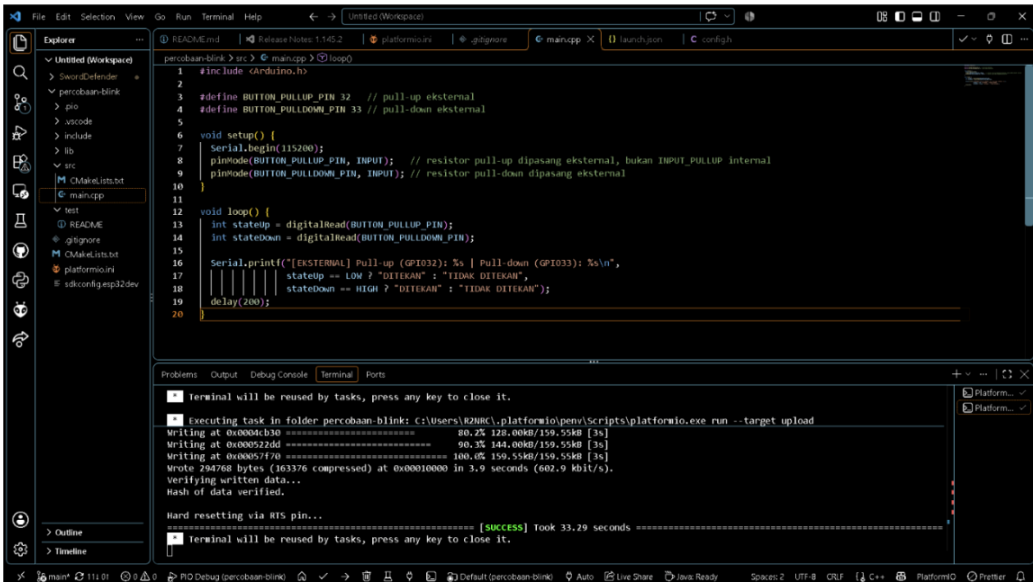
#define BUTTON_PULLUP_PIN 32   // pull-up eksternal
#define BUTTON_PULLDOWN_PIN 33 // pull-down eksternal

void setup() {
    Serial.begin(115200);
    pinMode(BUTTON_PULLUP_PIN, INPUT);
    pinMode(BUTTON_PULLDOWN_PIN, INPUT);
}

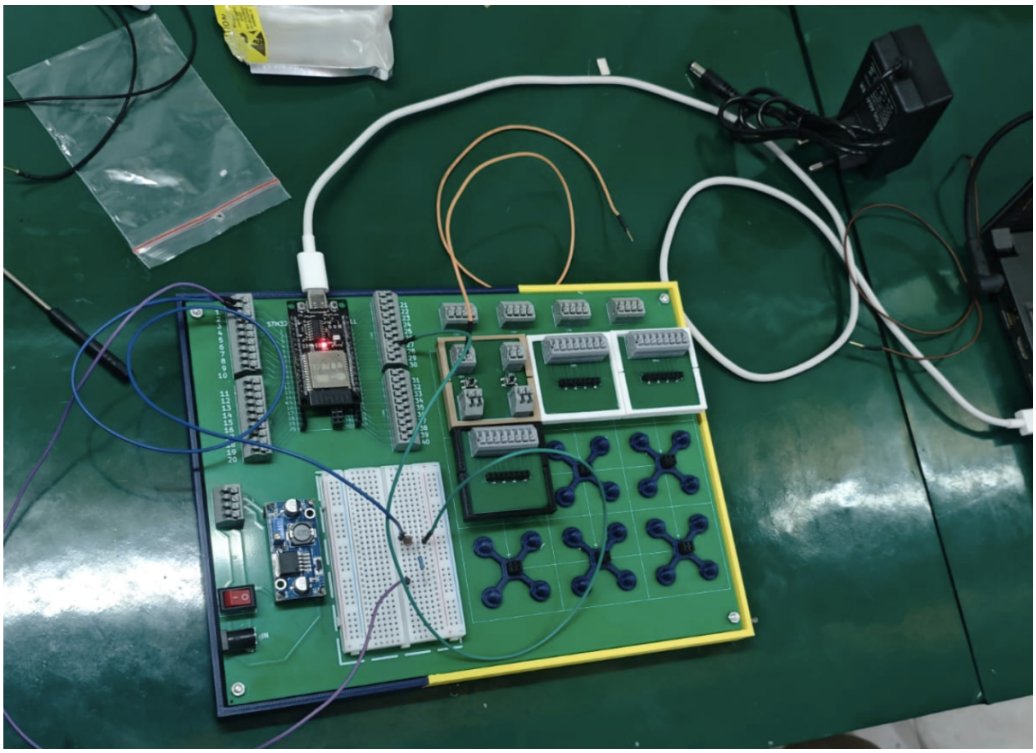
void loop() {
    int stateUp = digitalRead(BUTTON_PULLUP_PIN);
    int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

    Serial.printf("[EKSTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
        stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
        stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
    delay(200);
}
```

4. Melakukan Build dan Upload, kemudian menekan kedua tombol dan mencatat hasil dari Serial Monitor.



Gambar 3: Hasil Percobaan 3 Bagian A - Kode Program Pull-up & Pull-down Eksternal berhasil di-upload dengan status [SUCCESS]



Gambar 4: Dokumentasi rangkaian perangkat keras pada percobaan pull-up dan pull-down eksternal

5. **Bagian B - Pull-up/Pull-down Internal:** Melepas kedua resistor eksternal (tombol tetap terpasang), mengubah koneksi pushbutton 2 dari 3.3V ke GND sesuai skema internal.

6. Mengganti isi `src/main.cpp` dengan Kode Program B (Internal):

Listing 7: Kode Program B - Pull-up & Pull-down Internal

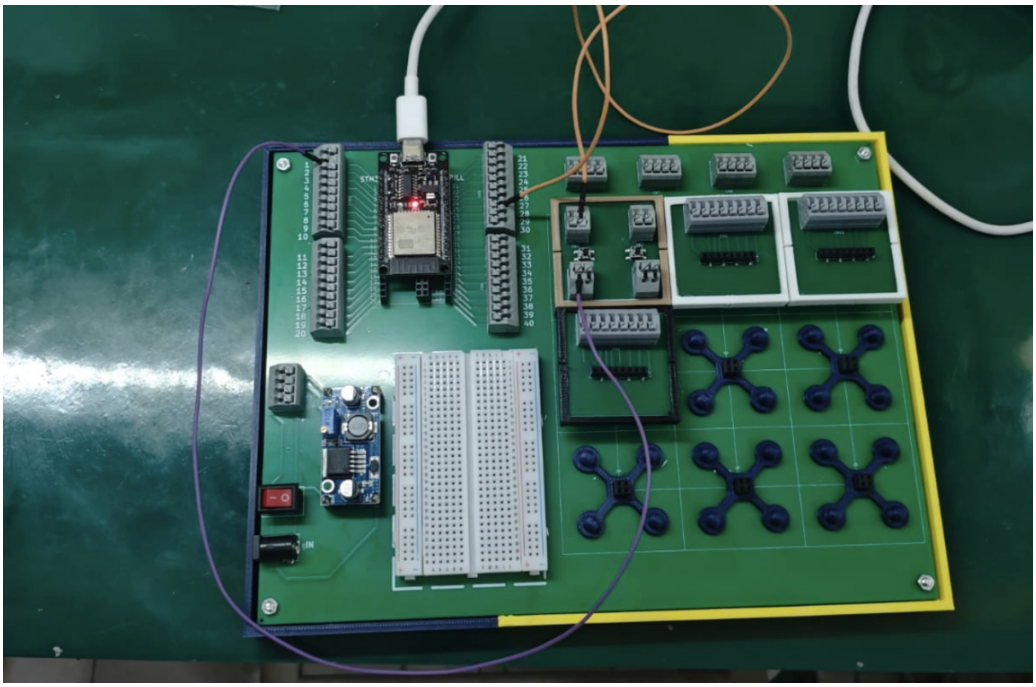
```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32   // pull-up internal
#define BUTTON_PULLDOWN_PIN 33 // pull-down internal
```

```
5
6  void setup() {
7      Serial.begin(115200);
8      pinMode(BUTTON_PULLUP_PIN, INPUT_PULLUP);
9      pinMode(BUTTON_PULLDOWN_PIN, INPUT_PULLDOWN);
10 }
11
12 void loop() {
13     int stateUp = digitalRead(BUTTON_PULLUP_PIN);
14     int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);
15
16     Serial.printf("[INTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
17         stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
18         stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
19     delay(200);
20 }
```

7. Melakukan Build dan Upload, kemudian menekan kedua tombol dan mencatat hasil dari Serial Monitor.

8. Membandingkan hasil Bagian A dan Bagian B. Hasil yang diperoleh seharusnya sama: pull-up bernilai HIGH saat tidak ditekan dan LOW saat ditekan, pull-down sebaliknya.



Gambar 5: Dokumentasi rangkaian pengujian konfigurasi resistor internal ESP32

1.4 Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

Tujuan: Mengidentifikasi permasalahan bouncing pada tombol mekanik dan mengimplementasikan debouncing berbasis software pada ESP32 dengan framework Arduino.
Langkah Kerja:

1. Memasang satu pushbutton pada GPIO 25 ESP32 dengan konfigurasi pull-up internal (satu kaki ke GPIO, kaki lain ke GND).
2. Konfigurasi `platformio.ini` tetap sama seperti Percobaan 3 (ESP32 Arduino), sehingga tidak perlu diubah.
3. Menulis kode program perbandingan counter tanpa debouncing dan dengan debouncing pada `src/main.cpp`:

Listing 8: Kode Program Debouncing

```
#include <Arduino.h>

#define BUTTON_PIN 25
#define DEBOUNCE_DELAY 50 // ms

// Tanpa debouncing
int rawLastState = HIGH;
unsigned long counterNoDebounce = 0;

// Dengan debouncing
int lastFlickerableState = HIGH;
int stableState = HIGH;
unsigned long lastDebounceTime = 0;
unsigned long counterWithDebounce = 0;

void setup() {
    Serial.begin(115200);
    pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
    int reading = digitalRead(BUTTON_PIN);

    // ===== Tanpa Debouncing =====
    if (reading == LOW && rawLastState == HIGH) {
        counterNoDebounce++;
    }
    rawLastState = reading;

    // ===== Dengan Debouncing =====
    if (reading != lastFlickerableState) {
        lastDebounceTime = millis();
    }
    if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY) {
        if (reading != stableState) {
            stableState = reading;
            if (stableState == LOW) {
                counterWithDebounce++;
            }
        }
    }
    lastFlickerableState = reading;

    // ===== Tampilkan kedua nilai =====
```

```
45      static unsigned long lastPrintedNoDebounce = 0;
46      static unsigned long lastPrintedWithDebounce = 0;
47      if (counterNoDebounce != lastPrintedNoDebounce || counterWithDebounce !=
        lastPrintedWithDebounce) {
48          Serial.printf("Tanpa Debounce: %lu\t Dengan Debounce: %lu\n",
        counterNoDebounce, counterWithDebounce);
49          lastPrintedNoDebounce = counterNoDebounce;
50          lastPrintedWithDebounce = counterWithDebounce;
51      }
52  }
```

4. Melakukan Build dan Upload, kemudian menekan tombol satu kali dan membandingkan nilai kedua counter di Serial Monitor.

5. Mengulangi penekanan dengan cara berbeda (cepat dan lambat) untuk mengamati efek bouncing. Counter tanpa debouncing dapat naik lebih dari 1 untuk satu tekanan, sedangkan counter dengan debouncing naik tepat 1.

6. Mengubah nilai `DEBOUNCE_DELAY` menjadi 10 ms, kemudian 200 ms, lalu melakukan Build dan Upload untuk masing-masing nilai dan membandingkan ketiga hasil tersebut.

Gambar 6: Hasil Percobaan 4 - Kode program debouncing berhasil di-upload dengan status [SUCCESS], menampilkan variabel counter pada Serial Monitor

1.5 Percobaan 5: Akses GPIO Analog - Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

Tujuan: Memahami ADC sebagai mode akses GPIO ketiga (analog input) dan mengimplementasikan pembacaan nilai analog sederhana menggunakan LDR.
Langkah Kerja:

1. Memasang LDR bersama resistor $10\text{k}\Omega$ dalam konfigurasi pembagi tegangan: satu kaki LDR ke 3.3V, kaki lain ke GPIO 34 dan ke GND melalui resistor $10\text{k}\Omega$.

2. Konfigurasi `platformio.ini` tetap sama (ESP32 Arduino), ditambahkan `monitor_speed = 115200` untuk Serial Monitor.

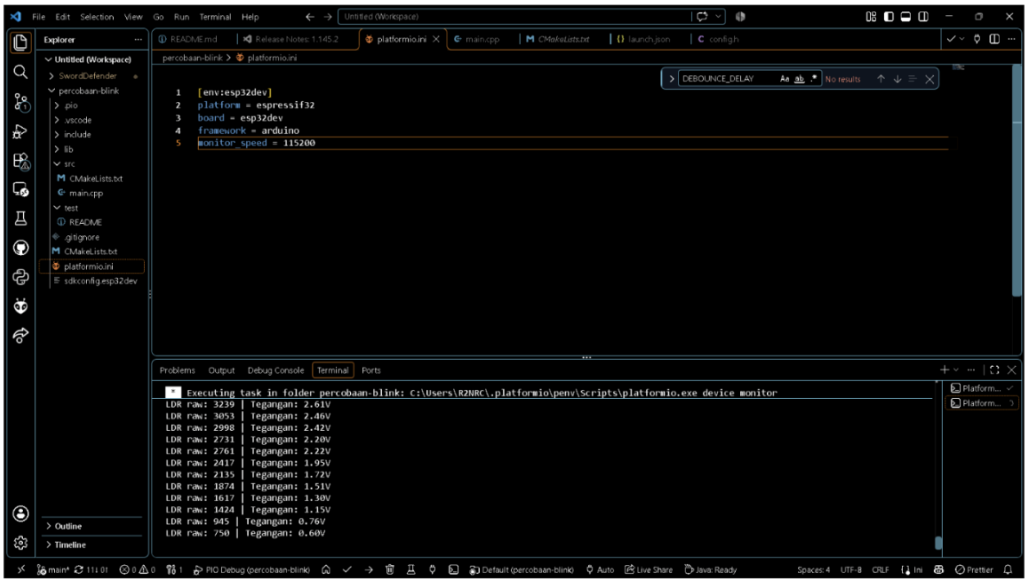
3. Menulis kode program pembacaan ADC pada `src/main.cpp`:

Listing 9: Kode Program Pembacaan ADC dengan LDR

```
1  #include <Arduino.h>
2
3  #define LDR_PIN 34
4
5  void setup() {
6      Serial.begin(115200);
7  }
8
9  void loop() {
10     int raw = analogRead(LDR_PIN);
```

```
    float voltage = raw * (3.3 / 4095.0);
    Serial.printf("LDR raw: %d | Tegangan: %.2f\n", raw, voltage);
    delay(300);
}
```

4. Melakukan Build dan Upload, kemudian membuka Serial Monitor.
5. Menutup LDR dengan tangan (nilai ADC menurun, tegangan turun) dan menyinari dengan lampu (nilai ADC naik, tegangan naik). Nilai mentah berubah dalam rentang 0-4095.



Gambar 7: Hasil Percobaan 5 - Pembacaan ADC dengan LDR pada Serial Monitor menunjukkan nilai raw dan tegangan yang berubah sesuai intensitas cahaya

2 Analisis Hasil Percobaan

Pelaksanaan rangkaian praktikum Modul 1 ini dimulai dengan pengenalan lingkungan pengembangan berbasis Visual Studio Code dan PlatformIO, di mana proses kompilasi serta pemindahan program berhasil dilakukan pada dua arsitektur mikrokontroler yang berbeda, yaitu STM32 Blackpill dan ESP32. Pada Percobaan 1, program Blink sukses dijalankan di atas board STM32 Blackpill melalui perangkat programmer eksternal ST-Link menggunakan protokol SWD. Pengamatan menunjukkan bahwa pin onboard PC13 bekerja secara aktif rendah (*active LOW*), sehingga pemberian logika rendah memicu LED menyala secara periodik.

Selanjutnya, Percobaan 2 mengeksplorasi board ESP32 menggunakan framework resmi ESP-IDF yang berbasis Real-Time Operating System. Berbeda dengan struktur sekuensial sederhana pada Arduino, framework ESP-IDF menuntut penggunaan fungsi titik masuk *app_main()* serta penundaan berbasis RTOS seperti *vTaskDelay*. Meskipun membutuhkan waktu kompilasi dan ukuran berkas biner yang lebih besar akibat penyertaan pustaka sistem tingkat rendah, program berhasil mengontrol LED di pin GPIO 2 dengan baik setelah penyesuaian konfigurasi penargetan board diselesaikan.

Pada pengujian antarmuka pin digital melalui Percobaan 3, perbandingan antara resistor eksternal dan resistor internal (*internal pull-up* dan *pull-down*) memperlihatkan hasil keluaran yang identik dan stabil. Konfigurasi resistor internal terbukti efektif untuk menghilangkan potensi sinyal mengambang (*floating*) tanpa memerlukan tambahan komponen fisik di luar mikrokontroler. Kendati demikian,

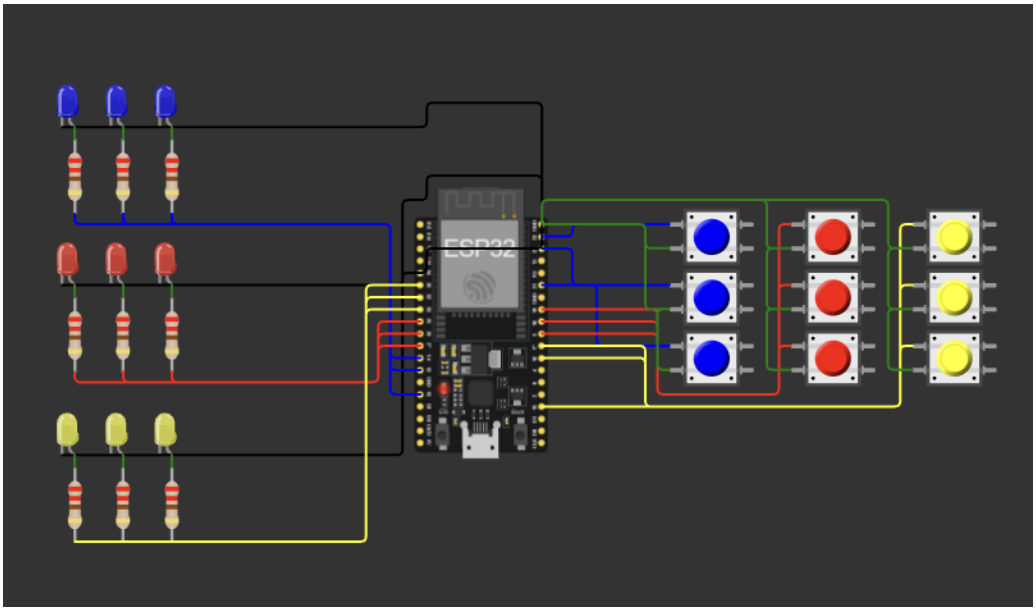
penggunaan tombol mekanik secara langsung tetap memunculkan tantangan fisik berupa fenomena pantulan kontak (*bouncing*). Hal ini dibuktikan secara empiris dalam Percobaan 4, di mana penghitungan pulsa tanpa algoritma peredam sering kali mencatat nilai ganda yang tidak valid untuk satu kali penekanan. Implementasi algoritma peredam perangkat lunak berbasis fungsi pewaktu *millis()* berhasil mengatasi gangguan tersebut, dengan parameter penundaan optimum sebesar 50 milidetik yang mampu menyeimbangkan ketepatan pembacaan sinyal dan tingkat responsivitas sistem.

Tahap akhir eksplorasi GPIO diperluas melalui Percobaan 5 dengan memanfaatkan kanal masukan analog *Analog-to-Digital Converter* (ADC) untuk membaca perubahan nilai resistansi sensor LDR. Integrasi rangkaian pembagi tegangan menunjukkan bahwa variasi intensitas cahaya lingkungan diterjemahkan secara linear ke dalam rentang nilai digital diskret 12-bit (0 hingga 4095). Penggunaan pin khusus masukan analog pada ESP32 memastikan proses konversi berjalan akurat tanpa mengganggu modul komunikasi nirkabel, sekaligus menegaskan keunggulan sistem analog dalam merepresentasikan besaran fisik secara kontinu dibandingkan sekadar mendeteksi status logika biner.

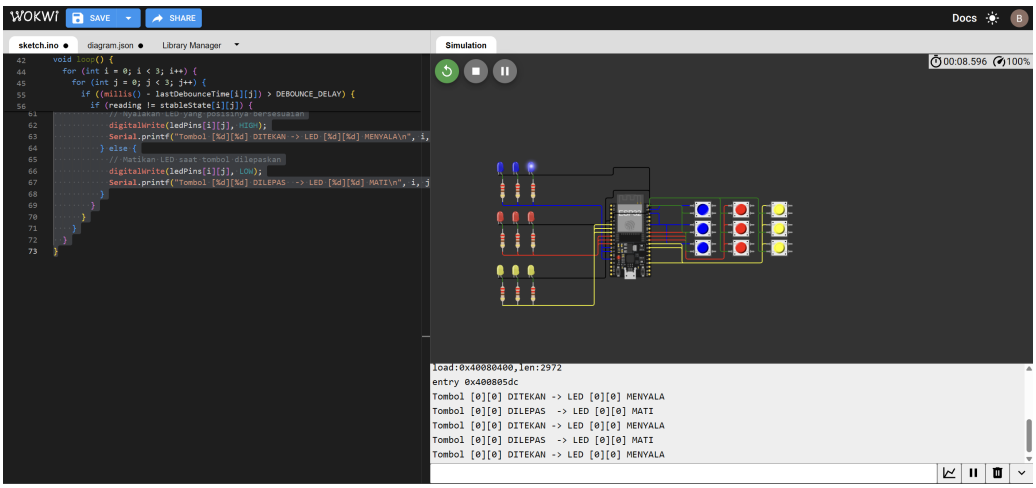
3 Hasil Tugas Modul

3.1 Simulasi Matrix Pushbutton 3×3 Mengendalikan Matrix LED 3×3

Bagian ini memuat hasil dari pengerjaan tugas tambahan yang diberikan dalam modul praktikum serta penjelasan dari jawaban tersebut. Simulasi dirancang menggunakan Wokwi dengan memanfaatkan ESP32 untuk membaca matriks tombol 3×3 dan mengontrol matriks LED 3×3 secara bersesuaian.



Gambar 8: Rangkaian keseluruhan simulasi Matrix Pushbutton 3×3 dan Matrix LED 3×3 pada Wokwi



Gambar 9: Pengujian penekanan salah satu pushbutton untuk mengendalikan LED pada posisi yang bersesuaian, disertai tampilan source code dan Serial Monitor

Berikut adalah kode program tanpa komentar yang digunakan pada simulasi di atas:

Listing 10: Source Code Simulasi Matrix Pushbutton dan LED 3×3

```
#include <Arduino.h>

const int ledPins[3][3] = {
  {13, 12, 14},
  {27, 26, 25},
  {33, 32, 4}
};

const int buttonPins[3][3] = {
  {23, 22, 21},
  {19, 18, 5},
  {17, 16, 15}
};

unsigned long lastDebounceTime[3][3] = {0};
int lastFlickerableState[3][3];
int stableState[3][3];
const unsigned long DEBOUNCE_DELAY = 50;

void setup() {
  Serial.begin(115200);

  for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
      pinMode(ledPins[i][j], OUTPUT);
      digitalWrite(ledPins[i][j], LOW);

      pinMode(buttonPins[i][j], INPUT_PULLUP);

      stableState[i][j] = HIGH;
      lastFlickerableState[i][j] = HIGH;
    }
  }
}
```

```
35
36 void loop() {
37   for (int i = 0; i < 3; i++) {
38     for (int j = 0; j < 3; j++) {
39       int reading = digitalRead(buttonPins[i][j]);
40
41       if (reading != lastFlickerableState[i][j]) {
42         lastDebounceTime[i][j] = millis();
43         lastFlickerableState[i][j] = reading;
44       }
45
46       if ((millis() - lastDebounceTime[i][j]) > DEBOUNCE_DELAY) {
47         if (reading != stableState[i][j]) {
48           stableState[i][j] = reading;
49
50           if (stableState[i][j] == LOW) {
51             digitalWrite(ledPins[i][j], HIGH);
52             Serial.printf("Tombol [%d][%d] DITEKAN -> LED [%d][%d] MENYALA\n", i, j,
     i, j);
53           } else {
54             digitalWrite(ledPins[i][j], LOW);
55             Serial.printf("Tombol [%d][%d] DILEPAS  -> LED [%d][%d] MATI\n", i, j, i,
      j);
56           }
57         }
58       }
59     }
60   }
61 }
```

3.2 Jawaban Pertanyaan Analisis

3.2.1 Pertanyaan 1: Mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton?

Debouncing tetap diperlukan karena fenomena bouncing merupakan karakteristik fisik murni dari kontak mekanik tombol itu sendiri, bukan dipengaruhi oleh topologi rangkaian matriksnya. Tanpa adanya algoritma peredam, getaran transien saat tombol ditekan atau dilepas akan terbaca oleh mikrokontroler sebagai serangkaian pulsa liar yang mengganggu kestabilan sistem logika.

3.2.2 Pertanyaan 2: Analisis konfigurasi pull-up/pull-down pada pembacaan tombol matrix

Penggunaan konfigurasi *internal pull-up* memastikan pin GPIO berada pada kondisi pasti bernilai HIGH saat tombol tidak terhubung ke ground. Hal ini krusial untuk mencegah kondisi mengambang (*floating*) yang rentan terhadap derau elektromagnetik, sehingga sistem terhindar dari pembacaan palsu (*phantom key press*).

3.2.3 Pertanyaan 3: Perbedaan digital input vs analog input (ADC)

Masukan digital hanya mengenali dua keadaan diskret (HIGH atau LOW) berdasarkan ambang batas tegangan tertentu. Sebaliknya, masukan analog melalui ADC mampu mengukur nilai tegangan kon-

tinu secara presisi dan mengubahnya menjadi rentang nilai digital multitingkat (seperti 0 hingga 4095 pada resolusi 12-bit), memungkinkan pembacaan sensor intensitas cahaya secara halus.

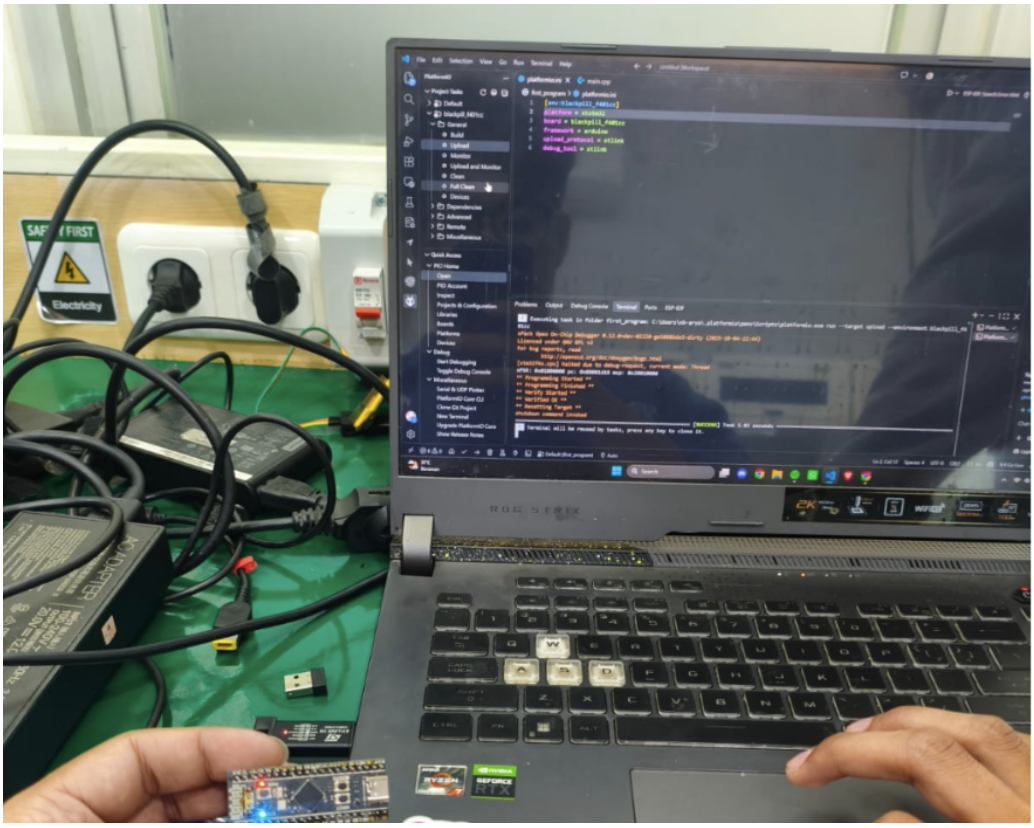
4 Kesimpulan

Secara keseluruhan, rangkaian praktikum Modul 1 ini berhasil dilaksanakan dengan baik, dimulai dari tahap penyiapan perangkat lunak menggunakan Visual Studio Code dan PlatformIO hingga pengujian fungsionalitas perangkat keras pada arsitektur STM32 Blackpill dan ESP32. Berbagai eksperimen yang dilakukan membuktikan bahwa konfigurasi pin GPIO untuk keperluan keluaran digital, masukan tombol dengan resistor internal maupun eksternal, serta pembacaan kanal analog melalui ADC dapat berjalan secara stabil dan sesuai dengan landasan teori. Selain itu, implementasi peredam getaran mekanik (*debouncing*) berbasis perangkat lunak terbukti efektif dalam menghasilkan data masukan yang akurat, yang kemudian diperkuat melalui simulasi sistem matriks tombol dan lampu LED menggunakan platform virtual.

5 Lampiran

5.1 Dokumentasi saat praktikum

Menampilkan foto selama pelaksanaan praktikum. Dokumentasi meliputi foto alat yang digunakan dan foto praktikan saat praktikum sebagai bukti otentik pelaksanaan kegiatan.



Gambar 10: Dokumentasi pelaksanaan percobaan saat praktikum